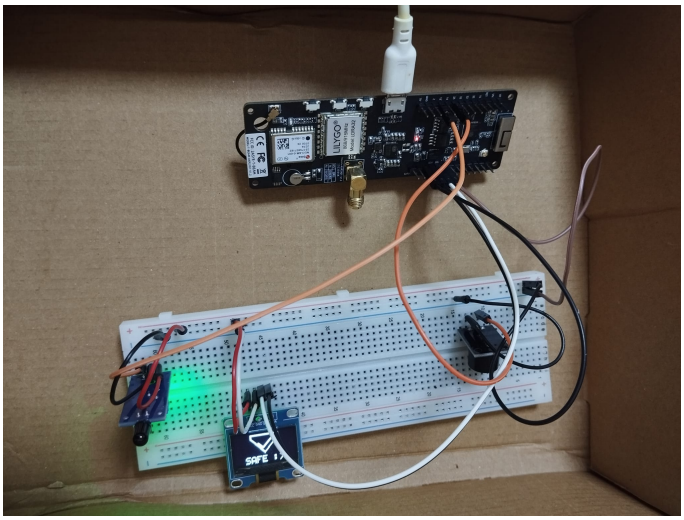
ESP32-Based Flame Detection and Real-Time Monitoring System

An Internet of Things (IoT) Based
Early Fire Warning Project



University	Recep Tayyip Erdoğan University
Department	Computer Engineering
Instructor	Joseph Bamidele Awotunde
Name	Yasin ENİŞ
Student No	221401043
E-mail	`yasin_enis22@erdogan.edu.tr`
Platform	LilyGO TTGO T-Beam (ESP32), ESP-IDF (C)

June 17, 2026

Abstract

This report describes a real-time **flame (fire) detection and remote monitoring** system built on a LilyGO TTGO T-Beam (ESP32) development board. The system continuously monitors the signal from an infrared flame sensor; when a flame is detected, it raises both an **audible** (active buzzer) and a **visual** (SSD1306 OLED display) alert locally. At the same time, the device broadcasts its status data over Wi-Fi using **WebSocket**, and a web control panel displays this data live: alarm status, power/battery, signal strength, uptime, alarm-history charts, an event log, and remote commands (mute / test / restart). All hardware components used in the project are introduced with photographs, their technical specifications are explained, and the wiring scheme, software architecture, and web interface are covered in detail.

Contents

1 Introduction

Fires are emergencies in which loss of life and property can be largely prevented when they are detected early. The goal of this project is to develop a flame detection system that provides an **early warning** using low-cost, widely available components, while also being **remotely monitorable over a network**.

Whereas traditional smoke/flame detectors only produce a local alarm, this system adopts an Internet of Things (IoT) approach: the device continuously streams its measured state to a web panel. As a result, even when the user is not physically next to the device, they can instantly view the alarm status, the device's health (battery, power source, signal strength, memory), and the history of past alarms; and they can send remote commands.

1.1 Key Features

- **Real-time flame detection:** The infrared flame sensor detects the IR radiation emitted by fire (GPIO13 digital input).
- **Audible alert:** The active buzzer (GPIO25) is triggered the instant a flame appears.
- **Visual alert:** The OLED shows a "Shield (SAFE)" icon normally and a full-screen "Flame (FIRE!)" icon during an alarm (pixel-level drawing via a framebuffer).
- **Power management:** The AXP-series PMU on the T-Beam is configured over I^2C; battery voltage/current and USB status are read.
- **Live web panel:** Status streaming, alarm-history charts, an event log, and remote control via WebSocket (primary) + HTTP polling (fallback).

2 System Architecture

The system consists of two main layers: (1) the embedded device in the field (the T-Beam based sensor node) and (2) the web monitoring panel running in the user's browser. The device manages the sensor and peripherals over I^2C and GPIO, and transmits its measured state to the panel in JSON form over Wi-Fi.

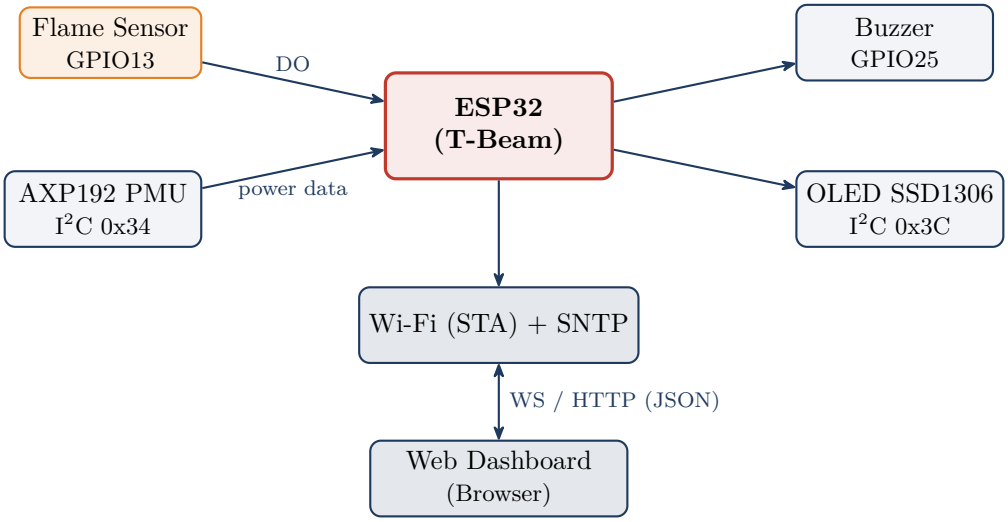


Figure 1: System block diagram: sensor input, peripheral outputs, and the network layer.

The actual assembly of the device is shown in Figure 2; the T-Beam board is connected with jumper wires to the sensor, buzzer, and OLED on the breadboard.

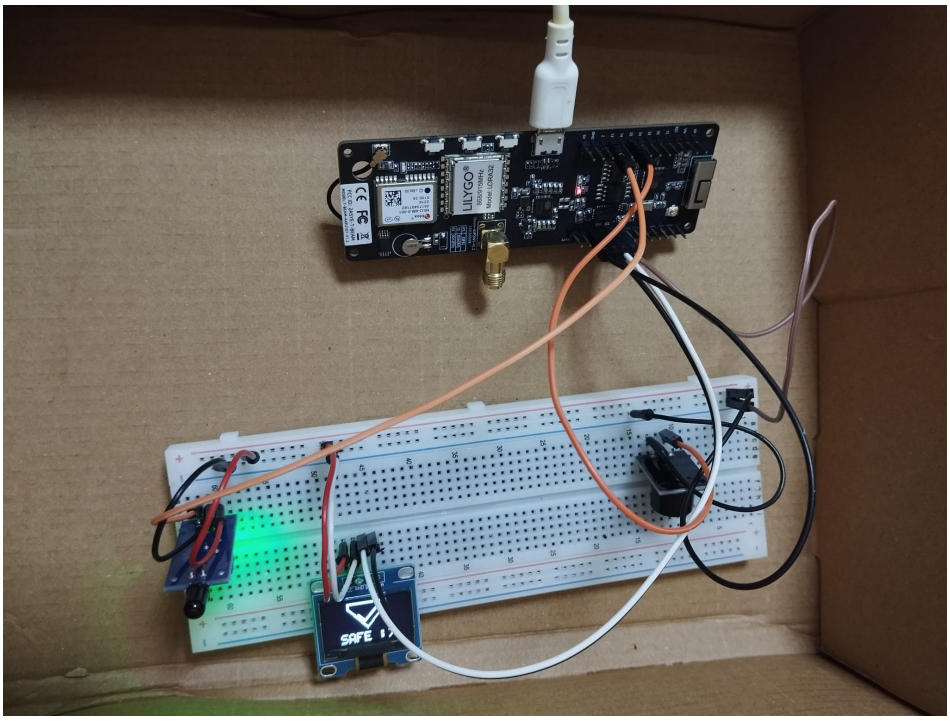


Figure 2: The completed system assembled on the breadboard. The OLED display shows the "SAFE" state.

3 Hardware Components

This section introduces each component used in the system with a photograph and explains its technical specifications and its role in the project.

3.1 LilyGO TTGO T-Beam (ESP32 Development Board)

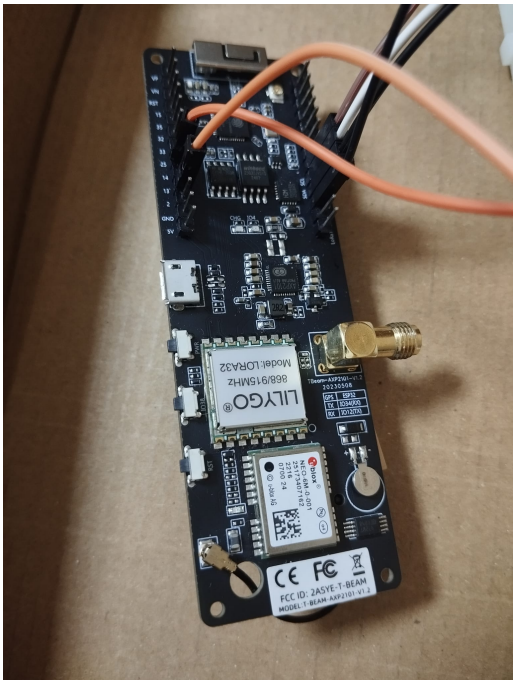


Figure 3: LilyGO TTGO T-Beam development board (ESP32 + LoRa + GPS + AXP PMU).

The brain of the project, the T-Beam, is a versatile development board carrying the Espressif **ESP32** processor. It includes several integrated units for communication and power management.

Table 1: LilyGO TTGO T-Beam technical specifications.

Feature	**Description**
Processor (MCU)	Espressif ESP32, dual-core Xtensa LX6, up to 240 MHz
Wireless	Wi-Fi 802.11 b/g/n + Bluetooth / BLE (integrated)
LoRa Module	Semtech SX127x based (868/915 MHz), with SMA antenna input
GPS	u-blox NEO-6M GNSS receiver
Power Mgmt. (PMU)	AXP-series PMU (firmware uses the AXP192 register map, I^2C 0x34)
Battery	18650 Li-ion holder on the back, built-in charging circuit
Interface	Micro-USB (power + programming), numerous GPIOs

Role in the project: This project uses the T-Beam's **ESP32 core**, its **Wi-Fi**, its **GPIOs**, the **I^2C** bus, and its **AXP192 PMU**. The PMU turns on the 3.3V power rails (LDO2/LDO3) that supply the OLED and provides battery/power telemetry. The LoRa and GPS units are not used in this project; they are left as potential future extensions.

3.2 Infrared Flame Sensor

Figure 4: Infrared (IR) flame sensor module; the black IR receiver, the LM393 comparator, and the sensitivity-adjustment potentiometer.

The flame sensor contains a photodiode that detects infrared radiation around 760–1100 nm emitted by fire. The LM393 comparator on the module compares this radiation against a threshold to produce a clean digital signal.

Table 2: Flame sensor module specifications.

Feature	Description
Detection principle	IR photodiode (sensitive to flame radiation, ∼760–1100 nm)
Comparator	LM393 (op-amp comparator)
Outputs	Digital (DO) and Analog (AO)
Sensitivity adjust	Threshold set via the on-board potentiometer
Detection angle	within a ∼60° cone
Supply	3.3V – 5V
Indicators	Power LED (PWR) and digital-output LED (DO)

Role in the project: The sensor's **digital output (DO)** is connected to the T-Beam's **GPIO13** pin. The firmware reads this pin every 200 ms; when a flame is detected, the output goes **LOW (0)** and the system enters the alarm state. The analog output (AO) is not used in this project.

3.3 HW-508 Active Buzzer Module

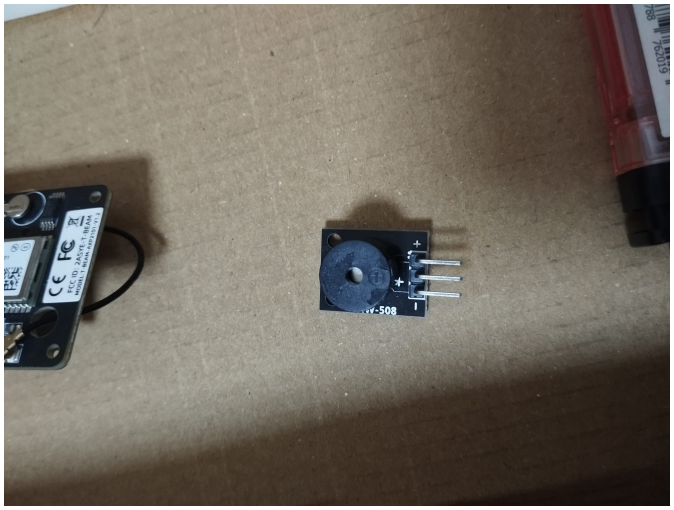


Figure 5: HW-508 active buzzer module.

The HW-508 is an **active** buzzer module with its own internal oscillator. Because of this, no PWM/frequency signal is needed to produce sound; it simply sounds when supplied with power.

Table 3: HW-508 active buzzer specifications.

Feature	Description
Type	Active buzzer (built-in oscillator)
Drive	DC supply is sufficient; no PWM required
Operating voltage	∼3.3V – 5V
Pins	(+) VCC, (–) GND, (S/I) signal

Role in the project: On this module, to avoid a continuous-beeping issue, the **VCC pin is connected directly to GPIO25** (active-high drive). When the ESP32 drives GPIO25 **HIGH**, power reaches the buzzer and it sounds; when **LOW**, it goes silent. The signal (S/I) pin is left

unconnected. This way the buzzer can be controlled directly by the flame state (and by the "mute" command).

3.4 0.96" OLED Display (SSD1306)

Figure 6: 0.96-inch I^{2}C OLED display (SSD1306, 128×64 pixels).

This OLED display with an SSD1306 driver is a self-illuminating (no backlight required), high-contrast monochrome display and is driven with just four wires (I^{2}C).

Table 4: OLED display specifications.

Feature	**Description**
Diagonal / resolution	0.96 inch / 128×64 pixels
Driver	SSD1306
Interface	I^{2}C (4 pins: VCC, GND, SCL, SDA)
I^{2}C address	0x3C
Supply	3.3V – 5V
Technology	OLED (self-illuminating, high contrast)

Role in the project: The display is connected to the T-Beam's I^{2}C bus (**SDA=GPIO21**, **SCL=GPIO22**). By drawing pixel by pixel into a 1024-byte **framebuffer**, the firmware produces two full-screen icons: a "Shield + Check (SAFE)" in the normal state and a "Flame (FIRE!)" during an alarm. The drawings are precomputed and pushed to the screen in a single transfer.

4 Wiring Scheme

All components share the common 3.3V power and GND rails over the breadboard. Table 5 summarizes each component's connection to the T-Beam.

Table 5: Pin connection table.

Component	Pin	T-Beam Connection	Note
Flame Sensor	VCC	3.3V	–
	GND	GND	–
	DO	GPIO13 (INPUT)	Digital signal; flame = LOW
	AO	Unused	Not used
HW-508 Buzzer	(+) VCC	GPIO25 (OUTPUT)	Active-high; HIGH = sounds
	(–) GND	GND	–
	S / I	Unused	Not used
OLED (SSD1306)	VCC	3.3V	via AXP192 LDO2
	GND	GND	–
	SDA	GPIO21	I^2C data
	SCL	GPIO22	I^2C clock

Important note: The VCC pin of the HW-508 active buzzer is connected directly to GPIO25, *not* to the 3.3V rail. This way the buzzer's power is switched directly by the GPIO and unwanted continuous beeping is prevented.

5 Software (Firmware)

The device firmware was developed in **C** using **ESP-IDF** (Espressif IoT Development Framework). The application runs on FreeRTOS and has a modular structure.

5.1 General Operation Flow

At startup, the I^2C bus, the AXP192 PMU (power rails + ADC), the OLED display, and the GPIOs are initialized in order; then Wi-Fi (STA) and SNTP (for a real timestamp) come up. The actual monitoring runs in a FreeRTOS task (`monitor_task`):

- The flame sensor (GPIO13) is read every 200 ms.
- On a state change (safe ↔ flame) the OLED is updated, a record is appended to the alarm history, and a WebSocket broadcast is sent **immediately**.
- The buzzer (GPIO25) is driven according to the flame state and the "mute" command.
- Even without a flame, a status broadcast is sent every 2 seconds to keep the panel "live".

5.2 OLED Framebuffer Drawing

The screen content is built pixel by pixel in a 128×64 = 1024-byte buffer (framebuffer). The "Flame" and "Shield" icons are drawn using row-by-row half-width tables and a custom 5×7 font; the ready buffer is pushed to the display in a single transfer over I^2C.

5.3 AXP192 Power Telemetry

By enabling the PMU's ADC channels, the battery voltage (12-bit), the charge and discharge current (13-bit), and the USB/charge status are read. If no battery is installed, the voltage and percentage read 0 (this is normal). These data are reflected on the panel as the power card.

5.4 Network Layer and Data Contract

When the device connects to Wi-Fi, it starts an HTTP/WebSocket server. The endpoints consumed by the panel are:

Table 6: Network endpoints served by the device.

Endpoint	Purpose
`ws://<IP>/ws`	Live status broadcast + remote command (primary)
`GET /api/status`	Instant status (HTTP fallback / polling)
`GET /api/alarms`	Alarm history (for charts and the log)
`POST /api/command`	Remote command (`mute`/`test`/`restart`)

The status data sent from the device to the panel conforms *exactly* to the following JSON schema:

```
{
  "deviceId": "tbeam-01", "online": true, "timestamp": 1718560000,
  "uptimeSec": 38211, "firmware": "1.0.0",
  "flame": { "detected": false, "sensorActive": true },
  "buzzer": false,
  "power": { "usbConnected": true, "charging": true,
    "batteryPercent": 87, "batteryVoltage": 4.05, "currentMa": 120 },
  "wifi": { "rssi": -58 },
  "system": { "freeHeap": 142000 }
}
```

6 Web Monitoring Panel (GUI)

The web panel is a static application written with HTML, Tailwind CSS, and vanilla JavaScript, using Chart.js for charts and requiring no build step. It connects to the device via **WebSocket**; if the connection drops, it automatically falls back to **HTTP polling**. The interface is bilingual (TR/EN) and supports dark/light themes.

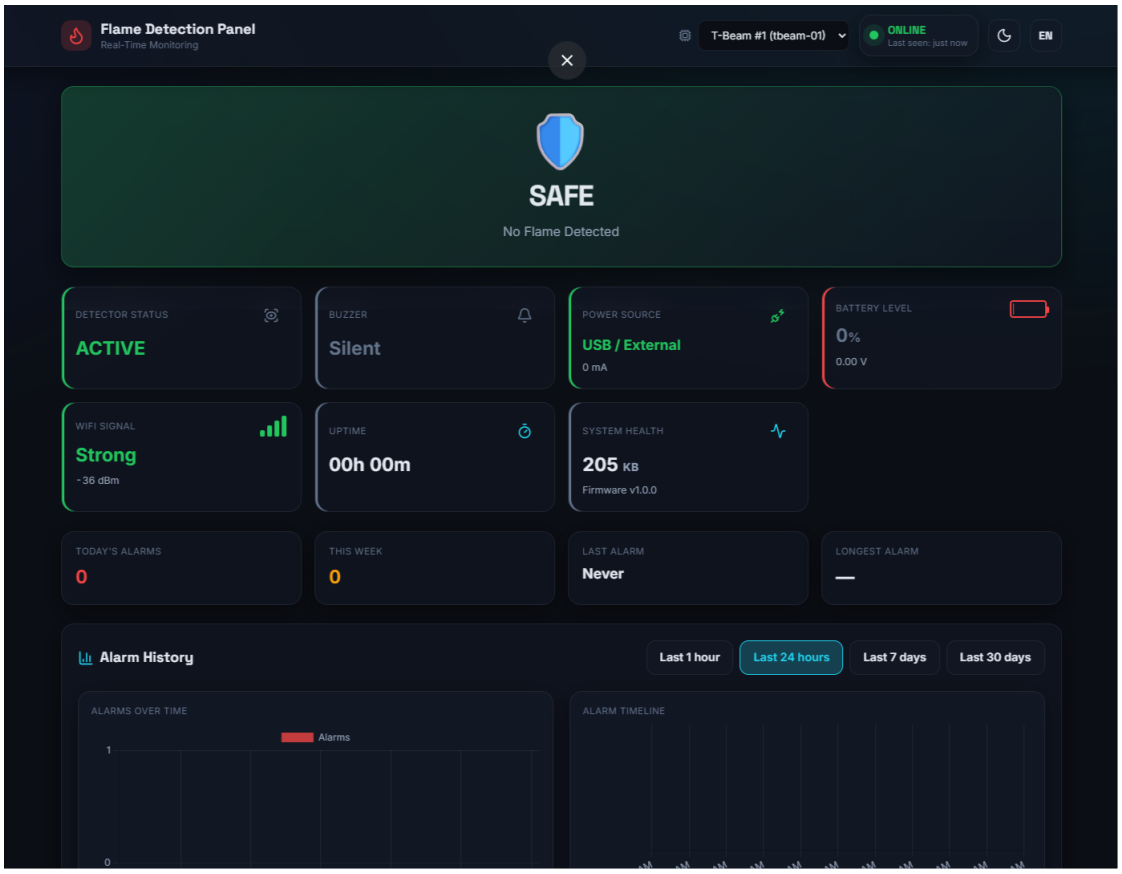


Figure 7: The web panel in the normal (SAFE) state. The main status panel is at the top; below it are the detector, buzzer, power source, battery, Wi-Fi signal, uptime, and system health cards; at the bottom are the alarm-history charts.

As shown in Figure 7, the panel presents all of the device's telemetry at a glance. When a flame is detected, the interface switches to the **red alarm** state in Figure 8: the main panel shows the "FLAME DETECTED!" warning, the buzzer card becomes "SOUNDING", the screen edge pulses red, and the audible/visual alerts are triggered.

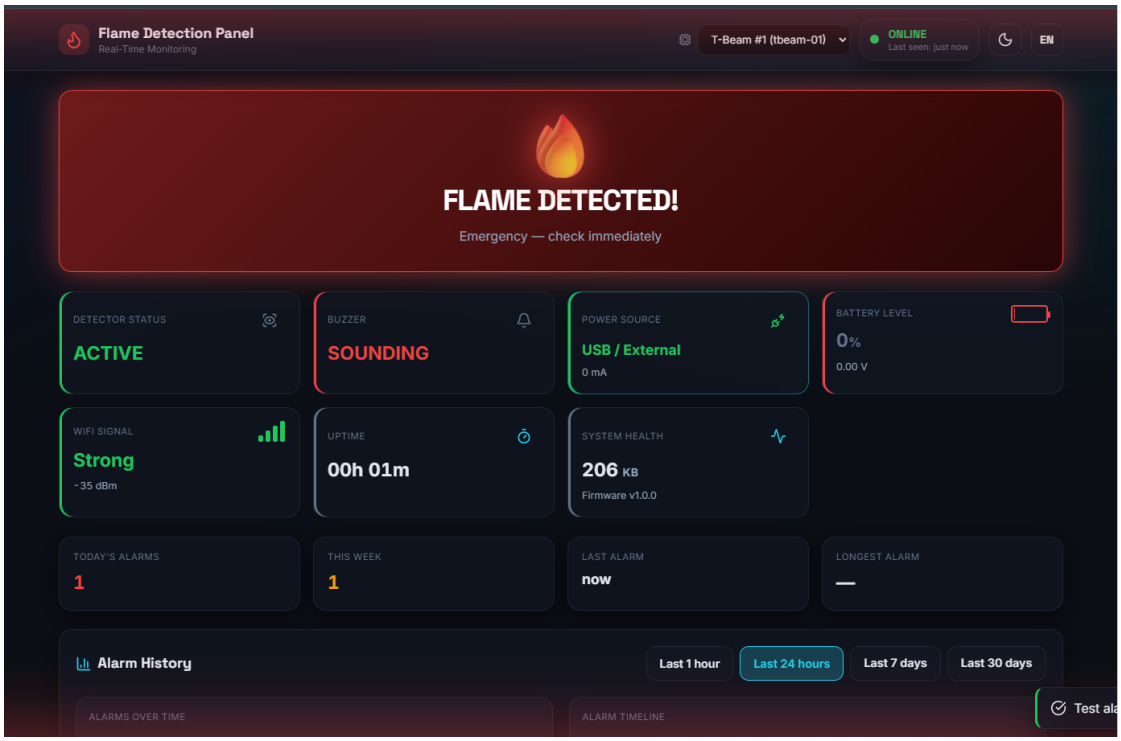


Figure 8: The web panel in the alarm (FLAME DETECTED) state. The buzzer is "SOUNDING", the day's alarm counter has increased, and "Last Alarm" has been updated to the current moment.

6.1 Panel Components

- **Main alarm panel:** Safe (green shield) / Flame (red, flashing).
- **Status cards:** Detector, buzzer, power source, battery (voltage/charge), Wi-Fi (RSSI bars), uptime, system health (free memory + firmware).
- **Statistics cards:** Today's/weekly alarm counts, last alarm (relative time), longest alarm.
- **Charts (Chart.js):** Alarms over time, alarm timeline, battery voltage and signal trend (1h / 24h / 7d / 30d ranges).
- **Event log:** Live row insertion, CSV export, "Acknowledge".
- **Remote control:** Mute the buzzer, test alarm, restart (with a confirmation modal).
- **Offline detection:** If no data arrives for a certain time, the device is automatically considered "OFFLINE" and the interface gracefully enters a waiting state.

7 Testing and Results

The system was tested both locally and on the network side:

- **Local alert:** When a flame source was brought close to the sensor, the buzzer sounded immediately and the OLED switched to the "FIRE!" icon. When the source was removed, the system returned to the "SAFE" state.
- **Live streaming:** The JSON broadcast by the device was verified over WebSocket; the `wifi.rssi`, `uptimeSec`, `system.freeHeap`, and `firmware` fields were observed to be reflected correctly on the panel.

- **Remote command:** A “test alarm” command sent from the panel triggered a temporary alarm on the device; this event was correctly processed into both the charts and the event log.
- **Robustness:** While the device was offline, the panel did not crash and entered the waiting state.

8 Conclusion

In this project, a flame detection system that works with low-cost, common components and can be monitored both locally (audibly/visually) and remotely over a network was successfully realized. While the ESP32-based T-Beam board provides early warning by managing the flame sensor, buzzer, and OLED display, it also becomes observable on a modern web panel through the real-time telemetry it broadcasts over Wi-Fi.

8.1 Future Work

- Long-range communication in areas without internet infrastructure using the **LoRa** module on the board.
- Adding the device location to the panel using **GPS** (mobile/field applications).
- Making **Telegram/E-mail** notifications real via a **backend**.
- **Multi-node** scenarios where several devices are monitored on the same panel.